

Министерство образования и науки Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра интеллектуальной обработки документов

Направление подготовки: 01.03.02 Прикладная математика и информатика
Направленность (профиль) подготовки: Прикладная математика, компьютерные
науки и инженерия

Выпускная квалификационная работа бакалавра

Исследование методов дообучения языковых моделей малого размера

Автор:

Студент 020 группы
Кратков Егор Вадимович

Научный руководитель:

Вопропаев Петр Михайлович



Москва 2024

Аннотация

Данная бакалаврская работа на тему: «Исследование методов дообучения языковых моделей малого размера» состоит из введения, двух глав, заключения. Главы разделены на параграфы, что позволяет сосредоточиться на отдельных задачах в рамках конкретного вопроса. Общий объем работы – 39 страниц, 3 таблицы, 22 источника литературы и 16 приложений.

Цель исследования - сравнить методы дообучения языковых моделей малого размера по качеству, времени, памяти и выявить лучшие.

Для достижения цели сформулированы следующие задачи: изучить теоретические основы языковых моделей малого размера и основной принцип работы языковых моделей малого размера, метод их дообучения для решения конкретных задач; разработать программу эмпирического исследования методов дообучения языковых моделей малого размера; провести эксперименты по дообучению с использованием разных методов; сравнить результаты по метрикам и сформулировать выводы по результатам исследования.

В ходе исследования нами выявлено, что адаптеры и современные методы дообучения дают результат, не хуже, а часто и лучше, чем классическое дообучение методом полной разморозки, используя при этом меньше ресурсов, такие как память и время.

Анализ полученных результатов позволил выявить лучшие методы для дообучения языковых моделей малого размера.

На основании данной работы нами предложены следующие рекомендации: использовать современные способов дообучения по типу: Parameter-Efficient Fine-Tuning методов.

Содержание

Введение	3
ГЛАВА 1. Теоретический подход к языковым моделям малого размера	5
1.1 Структура языковых моделей	5
1.2 Методы дообучения языковых моделей малого размера	15
Выводы по главе 1	20
ГЛАВА 2. Эмпирическое исследование методов дообучения языковых моделей малого размера	21
2.1 Исследование и построение решения задач.....	21
2.2 Выводы	32
Заключение	34
Список использованных источников	36
Приложение	39

Введение

Актуальность данной темы заключается в том, что дообучение моделей является одной из важных задач в современном машинном обучении. Эта область активно развивается, и постоянно изобретаются новые методы, которые, как авторы утверждают, превосходят предыдущие. Дообучение малых моделей актуально потому, что современные исследования, как правило, сосредоточены на больших моделях, поэтому важно сравнить их поведение при различных параметрах на малых моделях в условиях ограниченных ресурсов.

Объект исследования: языковые модели малого размера из BERT семейства.

Предмет исследования: методы дообучения языковых моделей малого размера.

Цель исследования: анализировать методы дообучения и выявить лучшие по качеству, времени, памяти.

Для реализации этой цели были поставлены следующие **задачи**:

- 1 Изучить теоретический основы языковых моделей малого размера и методы их дообучения.
- 2 Разработать программу исследования методов дообучения языковых моделей малого размера.
- 3 Провести эксперименты по дообучению с использованием разных методов и по подбору параметров различных методов.
- 4 Сравнить результаты по метрикам и сформулировать выводы по результатам исследования.

Методы исследования: метод эксперимента, измерения, сравнения, стандартная ошибка среднего, t-критерий Стьюдента.

Теоретическая и практическая значимость результатов состоит в предоставлении данных для применения методов дообучения предобученных моделей исходя из конкретных ограничений ресурсов, таких как видеопамять и время, в настройке параметров, а также в предоставлении готового кода для дообучения моделей, используя современную библиотеку huggingface.

Постановка задачи: нам дана языковая модель: около 100 млн параметров из семейства BERT моделей, которая предобучена на общее понимание языка. Наша цель - дообучить ее для решения конкретных задач.

Причем было решено сосредоточиться на методах, их параметрах и сравнении по разным метрикам и выбрать около четырех основных датасетов для экспериментов. В итоге:

- модель из BERT семейства: около 100 млн параметров
- 4-5 датасетов
- 3-4 метода дообучения
- сравнить по качеству, времени, нагрузке на GPU.

ГЛАВА 1. Теоретический подход к языковым моделям малого размера

1.1 Структура языковых моделей

Перед тем, как говорить о языковых моделях стоит обратиться к понятию нейронных сетей.

Согласно Большой российской энциклопедии - "Нейронные сети, искусственные, многослойные высокопараллельные (т. е. с большим числом независимо параллельно работающих элементов) логические структуры, составленные из формальных нейронов" [1].

Первая модель нейросети - перцептрон была разработана Фрэнком Розенблаттом в 1957 [2].

Согласно Сущене Р.В. и Кокаеву А.Э. идея нейронной сети - имитировать человеческий мозг. Однослойные нейронные сети содержат в своей основе один слой нейронов, получают на вход некоторые данные, затем обрабатывают их и выводят ответ. Многослойные нейронные сети - состоят из различных слоев однослойных нейронных сетей, и обрабатывают входные данные, пропуская через все слои.

Глубокие нейронные сети названы так, потому что содержат в себе большое количество слоев (более трех), а также часто более сложную структуру, похожую, например, на рекуррентные сети и сверточные сети [3].

Как писал Ксенофонов В.В. обучение с учителем таких нейронных сетей заключается в том, чтобы, имея как входные, так и выходные данные, которые мы хотим получить, обучить нейронную сеть. Основная идея - настроить параметры самой сети так, чтобы выходные данные были похожи на целевые. Причем мы хотим обучить модель так, чтобы это работало и на данных, которые наша модель не видела, но они приходят из того, же

распределения, что и данные, на которые модель обучается. Важно отметить, что должны присутствовать верные выходные данные, на которые мы планируем обучаться для того, чтобы это считалось обучение с учителем [4].

В нашей задаче мы дообучаем уже готовую модель, предобученную иметь общее понимание языка, решать конкретные задачи, имея небольшое количество данных, обычно от 1000 до 10000 объектов, используя технику обучения с учителем. По материалам статьи: "A Survey on Large Language Models: Applications, Challenges, Limitations, and Practical Usage" большие языковые модели - модели, которые могут работать с естественным языком и как правило обучались на огромном количестве данных [5].

Дополнительно стоит упомянуть основные концепции работы с естественным языком. В статье "Natural language processing: state of the art, current trends and challenges" кратко описываются основные принципы. NLP (Natural Language Processing) - обработка естественного языка. В ней можно выделить 2 ключевые части: NLU (Natural Language Understanding) - понимание естественного языка и NLG (Natural Language Generation) - генерация естественного языка. Понимание естественного языка связано с пониманием лингвистики, ключевыми компонентами которой являются: "Фонология - относится к звуку, морфология - формирование слов, синтаксис - структура предложения, семантика и прагматика, которая связана с пониманием." [6]. Основными задачами в NLP являются: NLI (Natural Language Inference) - логическое следование. Примером такой задачи может быть два утверждения, для которых нужно ответить: следует ли первое из второго. Суммаризация - краткое содержание текста. PoS Tagging - определение части слов для каждого слова в предложении. Coreference resolution - задача кореференции, которая заключается в определении всех фраз, которые используются для одного объекта. Например, нужно определить к какому существительному относится слово "он". Машинный

перевод связан с переводом текстов, QA (Question Answering) - ответы на вопросы, генерации текста и т.д.

Для всех этих задач используется один похожий принцип работы с данными: данные на вход подаются в текстовом виде. Первый этап - токенизация - разбиение текста на входные части (токены), обычно это могут быть слова, буквы, слоги. Им сопоставляются вектора - эмбединги. Они уже подаются на вход основной модели. В конце она генерирует метку для классификации, число или набор чисел для регрессии или текст для генерации.

Важно отметить основные датасеты, которые используются в NLP. Для начала введем понятие корпус: "Корпусом считается собрание текстов одного или нескольких языков, связанных между собой определенными параметрами" [7]. Для разных задач есть свои классические корпуса, Diksha Khurana, Aditya Koli, Kiran Khatter в статье Natural language processing: state of the art, current trends and challenges перечислили основные из них: для Sentiment Analysis (анализ эмоциональной окраски) - SST (Stanford Sentiment Treebank), содержащий около 12000 предложений с обзоров фильмов. Для моделирования языка - WikiText-103, содержащий 103 млн токенов с Википедии. Для машинного перевода - WMT14 - содержащий пары переводов: английский-немецкий и английский-французский. Для ответов на вопросы: SQuAD (Stanford Question Answering Dataset) датасет с вопросами по статьям Википедии [8].

Одними из классических датасетов для обработки естественного языка стали GLUE и SuperGLUE бенчмарки (классический набор данных, основная цель которого, сравнить результаты разных моделей). GLUE - набор из 9 заданий, которые проверяют понимание языка моделями:

- SST-2 - датасет от Стэнфорда по проверке эмоциональной окраске, о котором рассказывалось ранее;
- WNLI - задача кореференции и следствия. Даны два предложения,

нужно понять, к верной ли сущности из первого текста относится высказывание во втором;

- CoLA - набор текстов, по которым нужно определить, являются ли они грамматически корректными;
- QQP - корпус, по которому нужно решить, являются ли два вопроса одинаковыми по смыслу;
- STS-B - набор пар предложений, по которым нужно предсказать схожесть;
- MNLI - набор пар текстов, в которых нужно предсказать, является ли второе следствием первого;
- MRPC - корпус, в котором нужно определить, являются ли два предложения одинаковыми по смыслу;
- QNLI - набор текстов, по которому нужно определить, содержит ли второй текст ответ на вопрос по первому;
- RTE - корпус, в котором нужно ответить, следует ли одно предложение из другого [9].

Так как с развитием больших языковых моделей улучшилось их качество, то они стали показывать очень высокие результаты на GLUE бенчмарке, поэтому был собран второй более сложный бенчмарк SuperGLUE, состоящий из более сложных 8 заданий: BoolQ - задача QA, которая состоит из текста и вопроса с ответом да/нет; CB - два текста, по которым нужно сказать, следует ли второе из первого. но первый текст больше второго, а в нем содержится вложенное предложение, по типу второго; COPA - содержит в себе текст, вопрос и два варианта ответа, нужно выбрать один из них; MultiRC - задача QA, в которой дан вопрос, набор ответов, нужно предсказать для них правда/ложь; ReCoRd - состоит из текста, запроса с замаскированной сущностью. Нужно определить сущность; RTE - рассказывалось ранее в GLUE; WiC - проверка на омонимию, нужно ответить, употребляется ли слово

в двух предложениях в одинаковом контексте; WSC - задача кореференции, в которой нужно определить к какой из списка сущностей обращается в тексте[10].

Теперь вернемся к большим языковым моделям. Первые модели начали появляться в 1950 - 1960 годах, но так как тогда машинное обучение не было развитым, то они работали на основе лингвистических правил. Затем в 1980-1990 появились модели, которые использовали вероятностные методы.

Позже с развитием нейронных сетей, появились большие языковые модели на базе рекуррентных сетей (RNN), идея которых заключается в том, что у них есть контекст, на вход мы подаем слова, которые вместе со старым контекстом преобразуются в новый, мы можем повторить данную процедуру снова, таким образом модель может работать рекуррентно. Также ее можно обучить на генерацию текста, во время которого мы на первой итерации подадим на вход токен <START>, таким образом модель будет понимать, что надо начинать генерировать текст. А когда она сгенерирует <END>, закончить работу. Одной из больших проблем рекуррентных сетей было сложность запоминания длинных текстов, за счёт затухания градиента и того, что на всю программу использовался один контекст. Для решения этой проблемы придумали LSTM, который дополнительно использует долговременный контекст.

В 2014 была придумана Seq2Seq модель, которая представляет собой 2 LSTM. В начале текст подается в первую - энкодер, а затем контекст вторая LSTM работает с контекстом - декодер. Такая модель очень хорошо подходит для перевода. Авторы статьи смогли достичь 34.81 BLEU на датасете WMT14 для перевода [11].

В конце 2014 года для все той же проблемы, что и в классическом RNN - проблемы работы с большими текстами придумали способ, который называется - механизм attention: после того, как входные данные проходят

через энкодер, получаются аннотации: $(h_1, h_2, \dots, h_{(l = \text{количество слов})})$. Но вот работа декодера немного отличается: дополнительно к получению нового промежуточного состояния (t) декодера из старого. Старое состояние (t - 1) декодера подается на вход функции

$$e_{ti} = a(h_i, s_{t-1})$$

для каждого i из энкодера, которая может представлять собой скалярное произведение или несколько матриц, а затем, используя softmax:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_k \exp(e_{tk})}$$

получаются коэффициенты, с которыми можно просуммировать исходные аннотации энкодера.

$$c_t = \sum_k^l \alpha_{tk} h_k$$

Таким образом мы получили второй вектор, который можно подать на вход декодеру или сконкатенировать с выходным состоянием декодера на шаге t и использовать для предсказания [12].

Мы получили механизм внимания, который теперь использует не только старое состояние декодера для получения нового, но и все вектора энкодера, которые можно ассоциировать с каждым словом. Можно считать, что теперь модель также просматривает преобразованные входные слова и ей не нужно запоминать все в одном или нескольких векторах декодера.

Большим шагом для языковых моделей и вступлением в эпоху самых современных архитектур стала статья 2017 года под названием "Attention Is All You Need" и в которой была придумана модель трансформера, имеющего в своей основе механизм attention [13]. Трансформер, как и Seq2Seq состоит из

двух частей: энкодера и декодера. Энкодер состоит из шести последовательных одинаковых слоев. На вход подается эмбединги сложенные с Positional Encoding - специальным вектором, помогающим определить номер слова в тексте. Ключевой частью энкодера является механизм self-attention: каждый входной вектор разбивается на query, key, value путем домножения на соответствующие матрицы. Далее для каждого query считается скалярное произведение со всеми key векторами, делится на корень из размерности вектора, чтобы сохранить распределение (авторы упоминали, что это положительно влияло на результат) применяется softmax и с полученными коэффициентами складываются value вектора.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Причем в модели таких self-attention используется несколько - 8, затем - конкатенируется и матрицей переводится в первоначальную размерность. Это называется МНА (Multi-Head Attention). Так как у нас получились такие же размерности, то можно применить skip connection - сложить с входными векторами для лучшей проходимости градиента. Затем это направляется в Layer Normalization - работает аналогично BatchNorm, но вместо батча нормализует вдоль каждой компоненты вектора. Это авторы статьи кратко называют Add & Norm. Далее это пропускается через двуслойную сеть и Add & Norm. Декодер устроен похожим образом, но в отличие от энкодера в нем в начале используется маскированный self-attention: во время обучения мы подаем в него весь текст, но предсказывая i -е слово мы не можем знать $i + 1$, поэтому они зануляются маской. Также в декодере применяется encoder-decoder attention - где query берется из предыдущего слоя декодера, а key, value - из выхода энкодера. Итого слой энкодера: МНА-AddNorm-FeedForward-AddNorm. А декодера: MaskedМНА-AddNorm-EncoderDecoderAttention-

AddNorm-FeedForward-AddNorm.

Данная модель произвела прорыв в сфере больших языков моделей и машинного обучения в целом, т.к., во-первых, она показала одни из лучших результатов на задачах разного типа, например: 41.8 BLEU на датасете WMT14 по переводу с английского на французский. А во-вторых, для каждого query внутри одного слоя модель можно тренировать параллельно, что значительно ускоряет обучение.

Трансформеры стали использоваться повсеместно и породили две ключевых вариации: GPT (Generative Pretrained Transformer) от OpenAI, в основу которого лег декодер из трансформера и BERT (Bidirectional Encoder Representations from Transformers) от Google, которой основывался на энкодере. Так как для нашей задачи нужны будут модели по типу BERT, то на них остановимся поподробнее.

В 2018 году Google Ai выпустили модель BERT - Bidirectional Encoder Representations from Transformers [14].

В данной статье они взяли за основу энкодер из трансформера. В классической вариации BERT_{BASE} состоит из L=12 слоев, H=768 - размерность скрытого состояния, A=12 - голов self-attention, всего 110 млн параметров. Еще они обучили BERT_{LARGE} (L=24, H=1024, A=16, Total=340M) модель, которая имела в несколько раз больше параметров. Но в отличие от классического трансформера в BERT добавили некоторые изменения.

Во-первых, они уделили внимание входным данным и добавили несколько токенов: [SEP], с помощью которого можно разделять предложения, причем оставили поддерживаемость работы с одним предложением, так что этот токен не является обязательным. Таким образом, в BERT удобно подавать как одно, так и два предложения. Также ко входу добавляется токен [CLS], который можно использовать для классификации и регрессии, добавив, например, линейный слой.

Во-вторых, они использовали необычный способ для обучения. Оно состоит из двух этапов: предобучение и дообучение. Для предобучения использовались два типа задач: маскирование слов (MaskedLM) и "предсказание следующего предложения (NSP - Next Sentence Prediction). Для первой задачи 15% токенов помечались [MASK] (точнее 15% выбиралась для помечания [MASK], из них 80% помечалось, часть заменялась случайными или не менялась) и модель пыталась их предсказать. Хотя на этапе дообучения такая техника не используется, что немного меняет распределение входных данных, это позволяет более явно использовать тот факт, что энкодер в трансформере - двунаправленный.

Вторая задача (NSP) заключается в том, на вход подаются два предложения, где с вероятностью в 50% предложение "два" является следующим за предложением "один" а 50% - два случайных предложения из одного корпуса. Задача модели ответить - является ли второе предложение продолжением первого. Если внимательно посмотреть на обе эти задачи, то можно заметить, что для них не нужно собирать данные, как в классическом варианте обучения с учителем. Это позволило авторам использовать англоязычную Википедию (2500 млн слов) как один из корпусов для обучения. Можно предобработать данные, а затем запустить модель и она будет обучаться самостоятельно.

Для дообучения на разные задачи они разморозили BERT-а, а затем получали ответ с помощью [CLS] токена. В итоге, используя данную модель они получили лучший результат на всех 11 задачах из GLUE бенчмарка. В последующих годах многие исследователи пытались улучшить результаты, используя архитектуру, аналогичную BERT. И в 2019 году от Facebook

AI вышла модель RoBERTa (A Robustly Optimized BERT Pretraining Approach) - модель, которую мы будем использовать в экспериментах [15].

Они смогли улучшить результат BERT-а, не меняя архитектуру. Из

основных отличий можно выделить: использование нового датасета: CC-NEWS (76 GB), что почти в 5 раз больше, чем 16 GB датасетов BookCorpus и Википедии, которые использовались для предобучения BERT модели. Дополнительно авторы изменили маскирование. Авторы BERT-a создали маску один раз во время предобработки данных. Авторы RoBERTa маскировали каждый раз перед входом в модель, назвав это *dynamic masking* (динамическое маскирование) и получили небольшой прирост.

Во втором методе - NSP, они провели дополнительные эксперименты и решили полностью убрать эту часть, вместо это использовав метод *doc sentences*, где они брали тексты из одного документа и *full-sentences*, где дополнительно они начинали добавлять данные из второго документа, если первый закончится через [SEP] токен. Дополнительно они увеличили размер батча и размер словаря. Применив все эти изменения, они улучшили результат на каждой задаче из GLUE бенчмарка.

Новая волна улучшений была направлена на увеличение размера модели, особенно данную тенденцию можно проследить на примере GPT: у которой GPT-1 имела немного более 100 млн параметров, GPT-2 - 1.5 млрд, GPT-3 - 175 млрд, а GPT-4 - предположительно, более 1 трлн. Но мы сосредоточимся в данной работе на малых моделях в около 100 млн параметров, так как такие модели активно используются в случае, когда нет достаточной мощности для обучения моделей с более, чем 1 млрд параметров. BERT_{BASE} и RoBERTa_{BASE} можно считать стандартными примерами больших языковых моделей малого размера - около 100 млн параметров.

1.2 Методы дообучения языковых моделей малого размера

Как мы могли видеть: подход, в котором языковые модели в начале предобучаются на большом количестве данных, чтобы понимать общую структуру текста, а затем дообучаются под конкретные задачи, достаточно неплохо себя показывает. В современном машинном обучении такой подход приобрел большую популярность, так как не нужно обучать модель с нуля, а можно уже взять готовую и дообучить под свою задачу. Но с увеличением размера моделей специалисты столкнулись с проблемой: дообучать модели затратно по времени и памяти, поэтому исследователи начали искать новые методы, которые работают не хуже классического дообучения.

Техники дообучения можно разделить по типу задачи: модель можно дообучать на одной задаче, сразу на несколько задач, некоторые модели можно дообучать, используя всего лишь несколько примеров (few-shot fine-tuning) и zero-shot fine-tuning, при котором модель хотят дообучить решать задачи, которые она никогда не видела. В нашей работе мы сосредоточимся на классическом дообучении с учителем.

Существует довольно много методов дообучения моделей под свою задачу: например, можно настраивать гиперпараметры. Есть техника, использующая аугментацию данных. В области компьютерного зрения она хорошо себя показывает, в сфере NLP популярные методы аугментации включаютв себя: замену слов на синонимы, добавление ошибок, замена случайных слов и букв, перевод на другой язык и обратно, генерация с помощью моде лей. Довольно подробно они описаны в работе "A Comprehensive Study onNLP Data Augmentation for Hate Speech Detection: Legacy Methods, BERT, and LLMs" [16].

Авторы получали прирост в несколько процентов, используя данный

метод. Из более современных подходов можно выделить использование дообучения с подкреплением. RLHF (Reinforcement Learning from Human Feedback) - техника, в которой дообучение происходит на основе обратной связи от человека, например: модель выдает два ответа, а человек выбирает тот, который ему лучше подходит. Но то, что объединяет эти методы — это то, что они размораживают модель полностью, в то время как эти методы можно разделить по тому, какие параметры будут дообучаться. И это важно, так как позволит использовать меньше ресурсов и получать, как показывает практика, достаточно высокие результаты. Именно на эти методы мы обратим основное внимание в нашей работе.

Vladislav Lialin, Vijeta Deshpande, Anna Rumshisky в статье "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning" разобрали основные методы дообучения с использованием меньшего количества параметров [17]. Такие методы называются Parameter-Efficient Fine-Tuning (PEFT): эти методы можно разделить на несколько групп:

Аддитивные методы: основная идея - добавить новые параметры и тренировать только их. Примерами таких методов могут являться адаптеры - небольшие сети, которые вставляются между трансформерами. Их улучшенные вариации, такие как AdaMix - использует вместо одного адаптера несколько, подставляя один каждый раз для разного входа. После обучения они усредняются. Проблемой такого метода является использование большого количества памяти.

В 2022 году вышел метод под названием (IA)³. Идея этого метода состоит в том, чтобы видоизменять вектора, путем поэлементного домножения на добавленный адаптер. Авторы добавляют его к value, key и результату функции активации после первого линейного слоя в FeedForward части в трансформеры. Особенностью этого метода является то, что он использует всего лишь 0.02% параметров от исходной модели.

Они протестировали этот метод на модели в 3 млрд параметров, и получили результаты, превосходящие лору (о ней будет сказано немного позже). Также большим преимуществом является то, что после обучения эти вектора можно слить с матрицами, кроме FeedForward, и этот метод будет работать почти за такое же время. Также к ним можно отнести prompt подход: prompt методы без дообучения предлагают изменять входной текст. Но существуют методы с дообучением: Prompt tuning добавляет матрицу ко входу. Но, как правило, он начинает хорошо работать на больших моделях и современные методы показывают результаты лучше. Но можно добавлять такие матрицы не только ко входу: Prefix-tuning использует добавление матриц в начале каждого слоя.

Селективные методы: основная идея таких методов состоит в том, что размораживаются не все параметры. Например, можно разморозить только несколько последних слоев трансформера, или только bias параметры- BitFit.

Методы репараметризации: идея состоит в том, чтобы репараметризовать параметры малоранговые пространства. Одним из самых популярных таких методов является LoRa, идея которого - разбить матрицу размерности (n, m) на произведение двух матриц: $(n, r) \times (r, m)$. А далее можно полностью заморозить модель, обновлять такие адаптеры и в конце сложить с оригинальной матрицей, получив таким образом модель такого же размера. LoRa показала достаточно хорошие результаты, набрала высокую популярность и породила различные модификации. Например, несколько популярных вариантов: KronA, использующая в себе кронекерово произведение и LoHa, которая разбивает в более низкоранговые матрицы, чем LoRa, а затем собирает в одну с помощью адамарового (поэлементного) произведения. Одним из методов, которые достигли высокие результаты, является AdaLoRa, которая

перераспределяет параметры в LoRa-методе так, что более важные параметры учитываются сильнее [17].

Самой большой популярности достигла LoRa, а также очень высокий результат показывает AdaLoRa, поэтому мы остановимся на них поподробнее LoRa. В 2021 году вышел PEFT (Parameter-Efficient Fine-Tuning) метод - LoRa[18]. Их подход основывался на исследовании Armen Aghajanyan, Luke Zettlemoyer, Sonal Gupta "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning" в котором авторы показали, что, часто существует низкоранговая репараметризация, позволяющая дообучить большие модели на конкретные задачи. Например, можно, изменив 200 параметров, а затем спроектируя обратно в пространство, дообучить RoBERTa для 90% на mrpc [19].

Сам метод состоит в том, чтобы матрицу разбить на произведение двух с рангом r (описывалось выше). Таким образом, получается репараметризация в матрицу ранга не больше r , как ранг произведения матриц.

$$h = W_0x + \Delta Wx = W_0x + BAx$$

Для матрицы W_0 предложенный метод замораживает W_0 и отдельно добавляет матрицы A и B . Причем в начале эксперимента B заполняется нулями, а A из гауссовского распределения, поэтому на первом шаге дельта $W = 0$ и LoRa никак не влияет на модель.

Одним из больших преимуществ LoRa является то, что после обучения можно сложить данные адаптеры с оригинальной моделью и во время предсказаний она будет работать с такой же скоростью. Также одним из плюсов является то, что можно применять несколько адаптеров и использовать параллельно с другими методами, так как они отдельно

присоединяются к модели. В оригинальной статье LoRa исследуется только для матриц из self-attention для удобства. Также они заявляют, что использование VRAM GPU уменьшается и становится 1/3 от использования для дообучения методом полной разморозки. Замерив на GPT-3, скорость увеличилась на 25%. В оригинальной статье на модели GPT-3 они проверили то, к каким матрицам лучше применять из self-attention и получили, что ко всем матрицам дает наибольший результат, даже с меньшим рангом. Проверив ранг, они получили, что на ранге 4 модель уже показывает неплохие результаты. Также, сравнивая матрицу, полученную с помощью LoRa с матрицей полной разморозке, они пришли к выводу, что LoRa, предположительно, "усиливает основные детали для конкретных задачи, которые были выучены, но не акцентированы во время предобучения" [18].

В 2023 году вышла статья: AdaLoRA: Adaptive Budget Allocation for Parameter- Efficient Fine-Tuning, в которой авторы описали новый метод – AdaLoRA [20]. В этой статье авторы решили использовать немного другой подход и опираться на метод SVD (сингулярное разложение). Вместо AB, они представляют матрицу в виде $U: (n, r)$, $S: (r, r)$, $V(r, m)$. S – диагональная. U и V – за счет дополнительной регуляризации приближены к ортогональным. Вторая идея, которую они используют: зануление маловажных параметров, за счет зануления элементов в S :

$$\Lambda_k^{(t+1)} = \mathcal{T}(\tilde{\Lambda}_k^{(t)}, S_k^{(t)}), \text{ with } \mathcal{T}(\tilde{\Lambda}_k^{(t)}, S_k^{(t)})_{ii} = \begin{cases} \tilde{\Lambda}_{k,ii}^{(t)} & S_{k,i}^{(t)} \text{ is in the top-}b^{(t)} \text{ of } S^{(t)}, \\ 0 & \text{otherwise,} \end{cases}$$

Здесь Λ – диагональная матрица – S – функция важности. t – шаг, k – матрица, к которой применяют, i – компонента Λ , b – budget. Функция важности основывается на том, насколько веса влияют на градиент. Авторы показали, что данный метод работает немного лучше, чем LoRa.

Выводы по главе 1

Для достижения поставленной цели был изучен теоретический подход к структуре языковых моделей малого размера. Изучены основные источники русскоязычной и зарубежной литературы. Анализируя данные, мы получаем, что были рассмотрены модели, предшествующие большим языковым моделям, такие как, классические нейронные модели, лингвистические и статистические языковые модели. Далее были рассмотрены языковые модели: рекуррентные сети, LSTM, Seq2seq. Были разобраны основные понятия машинного обучения в области NLP, стандартные типы задач, популярные датасеты, используемые для обучения и тестирования моделей. Была разобрана архитектура трансформера и современные большие языковые модели малого размера, основанные на ней, такие как BERT и RoBERTa.

Также были рассмотрены основные методы дообучения больших языковых моделей, как методы, которые оставляют дообучение всей модели, но изменяют, другие этапы, например, аугментация данных, так и методы, изменяющие обучаемые параметры. Из них можно выделить те, которые обучают только часть параметров, например: BitFit, а также, которые используют дополнительные параметры, например: prompt tuning, (IA)3. Также были подробно описаны классические методы: LoRa, AdaLoRa.

ГЛАВА 2. Эмпирическое исследование методов дообучения языковых моделей малого размера

2.1 Исследование и построение решения задач

В рамках задачи нам дана языковая модель: около 100 млн параметров из семейства BERT моделей, которая предобучена на общее понимание языка. Наша цель - дообучить ее для решения конкретных задач.

Причем было решено сосредоточиться на методах, их параметрах и сравнении по разным метрикам и выбрать около четырех основных датасетов для экспериментов. В итоге: модель из BERT семейства: около 100 млн параметров; 4-5 датасетов; 3-4 метода дообучения; сравнить по качеству, времени, нагрузке на GPU.

Объект исследования: языковые модели малого размера.

Предмет исследования: методы дообучения языковых моделей малого размера.

Цель исследования: анализировать методы дообучения и выявить лучшие по качеству, времени, памяти.

Для реализации этой цели были поставлены следующие задачи: изучить теоретический основы языковых моделей малого размера и методы их дообучения; разработать программу исследования методов дообучения языковых моделей малого размера; провести эксперименты по дообучению с использованием разных методов и по подбору параметров различных методов; сравнить результаты по метрикам и сформулировать выводы по результатам исследования.

Методы исследования: метод эксперимента, измерения, сравнения.

Для решения поставленных задач была взята модель из BERT семейства – RoBERTa_{BASE}, т. к. эта модель такого же размера, как и BERT, но за счет

техник, используемых для обучения, показывает себя лучше. Она имеет около 125 млн параметров с модулем для классификации (более точное число: 124'647'170).

Из датасетов были выбраны: MRPC (NLI – проверить 2 предложения на эквивалентность) – стандартный датасет: данные для обучения: 3.67 тыс., данные для валидации: 408. Достаточно несложный, на нем также доступны тестовые данные, точнее целевые метки, так как на других датасетах GLUE/SuperGLUE тесты– скрытые. Следующим был взят датасет CB (NLI из superglue на следствие второго из вложенного в первое) – небольшой датасет: данные для обучения: 250, данные для валидации: 56. На нем удобно тестировать теории. RTE (следствие второго из первого) – датасет посложнее: данные для обучения: 2.49 тыс., данные для валидации: 277, на котором можно проверить насколько хорошо модель понимает более сложные задания. BOOLQ (QA: ответы на вопросы да/нет) – датасет больше с другим типом задания: данные для обучения: 9.43 тыс., данные для валидации: 3.27 тыс. Эти датасеты являются более удобными, так как они сравнительно небольшие и примерно средней сложности, так как на сложных датасетах из SuperGLUE модели малого размера показывают себя относительно слабо. Данная модель и датасеты были взяты из библиотеки huggingface. Дообучать мы тоже решили, используя huggingface Trainer, но отнаследовавшись от него и получив Custom Trainer. В большинстве экспериментов модель дообучалась 10 эпох (для датасета MRPC и RTE – 10 эпох, CB – 20, BOOLQ – 5) а затем брался максимум по валидационной выборке. Но, например, на этапе сравнения, после отдельной настройки каждой модели, использовался немного другой подход: модель обучалась 10 эпох, находился максимум по валидационной выборке, сохранялась конфигурация такой модели, а затем ее пропускали через тестовую выборку. Batch size везде был равен 16, т. к. даже в случае, когда с некоторыми адаптерами модель можно использовать с большим

батчем, на длительных обучении даже с методами по типу: `torch.cuda_empty_cache()`, память могла переполняться. Для того, чтобы результаты можно было аккуратно сравнить между собой по времени, батч везде был одинаковый, хотя замеры нагрузки на GPU при батчах большего размера также проводились.

Эксперименты были разбиты на две основные части: в первой мы брали каждый метод и рассматривали его с разными параметрами и нашли наилучшие, а во второй части мы сравнили эти методы между собой.

Полная разморозка (Full Fine-Tuning): мы решили начать с классического метода дообучения и использовать его как бейзлайн. Оптимизатором был использован AdamW с `adam_beta1 = 0.9`, `adam_beta2 = 0.999`, `weight_decay=0.01`, `adam_epsilon=1e-8`. Первым шагом мы решили подобрать `learning_rate`. Был выбран линейный `learning_rate`, так как для более точной настройки лучше подходит не константный `learning_rate`, а линейный считается классическим вариантом, подходящим под данное требование. Одной из причин также является то, что одна из наших целей посмотреть на связь результата от параметров адаптеров и, взяв такой `learning_rate`, результаты будут более интерпретируемые. Также мы эмпирически оценили и взяли `warmup_steps` примерно в 10%, а точнее 200 шагов для 2300 шагов всего дообучения и `weight_decay` в 0.01. Далее будут проводиться более точные подборы для данных параметров. Для подбора `learning_rate` были проверены параметры `[5e-6, 1e-5, 2e-5, 5e-5, 1e-4, 5e-4, 1e-3]`, на нескольких сидах на датасетах MRPC, CB, RTE. Эксперимент показал, что при линейном `learning_rate`, лучшие результаты получаются на промежутке `[1e-5, 5e-5]` (Приложение 1), так как при слишком большом `learning_rate` модель, как правило, скатывается в выбор наиболее часто встречающейся метки. При слишком низком `learning_rate`, результаты немного хуже, но не критично, скорее всего такой результат связан по большей мере с тем, что `learning_rate` –

линейный и при использования других техник результат для низких `learning_rate` мог бы отличаться. Тем не менее, мы решили остановиться на линейном `learning_rate` и взять тот, где модель показывает пик по метрикам. Оценив результаты на всех датасетах, мы решили использовать `learning_rate`: $2e-5$ и $5e-5$ для полной разморозки.

Далее мы решили рассмотреть такие параметры, как `warmup_steps` – количество шагов, на который `learning_rate` будет постепенно линейно подниматься с `lr/warmup_steps` до `lr` (а затем линейно опускаться). А также рассмотреть `weight_decay` в AdamW, который добавляет штраф за большие веса. Довольно часто эти параметры считаются относительно важными при обучении. Мы выполнили `grid search` (поиск по сетке) по ним, взяв `warmup_steps` [0, 200, 500] и `weight_decay` [0, 0.01, 0.05, 0.1, 0.2]. (для датасета `cb` `warmup_steps` были взяты [0, 20, 50, 100]). Такие параметры можно аргументировать тем, что брать слишком большие параметры для `warmup_steps` и `weight_decay` скорее вредят модели, так как слишком сильно регуляризуют ее. Оценив результаты, мы пришли к выводу, что заметной разницы от `weight_decay` и `warmup_steps` не наблюдалось, немного лучше себя показал `weight_decay = 0.01` (Приложение 2). Поэтому мы решили продолжать с 0.01 и `warmup_steps` в 10% от общего количества шагов.

LoRa(lora/лора): далее мы решили перейти к адаптерам, так как одним из желаемых результатов – получить более экономный вариант дообучения, чем классическая разморозка. Первоначальная конфигурация содержала в себе `r = 8` (ранг матриц), `alpha = 8` (добавление результата `lora` к замороженной матрице идет с коэффициентом `alpha/r`), `target_modules = [“query”, “value”]` (модули, к которым применяют `lora`), `lora_dropout = 0.1` (`dropout` для матриц `lora`), `bias=“none”` (дополнительное изменение `bias`, (`bias` модели, согласно `huggingface`), `modules_to_save=[“classifier”]`(дополнительный модуль для изменения, так как мы добавляем голову дополнительно и она заполняется

случайными числами, то мы решили эту часть модели разморозим). Мы решили более подробно сосредоточиться на rank, alpha и target_modules, так как считается, что они сильнее всего влияют на результат. Но начать мы решили с learning_rate: проведя аналогичный эксперимент мы получили похожие результаты: $5e-4$ на MRPC и CB и $7e-4$ на RTE показывают себя лучше всего (Приложение 3). Далее мы решили приступить к рассмотрению rank и alpha. Существует довольно много теорий: в оригинальной статье авторы писали, что на GPT-2 $r=4-16$ показывает себя лучше, но в целом уже с ранга 4 получаются неплохие результаты. Microsoft в своей имплементации lora использовали $r=8$, $a=16$ для RoBERTa_{BASE}. Также в Sebastian Raschka из lightning ai пришел к выводу, что $\alpha=2r$ работает лучше. Но там он использовал модель Llama 2 7 млрд параметров [21].

Также в статье “QLORA: Efficient Finetuning of Quantized LLMs” они нашли, что, начиная от ранга 8 особой разницы – нет, а alpha может быть и больше, чем ранг [22]. Некоторые приходили к выводам, что если rank и alpha – больше, то результат немного улучшается. Как мы видим, результаты различаются. Причем, согласно Aghajanyan. A. модели большего размера обычно имеют меньший intrinsic dimension по сравнению с размером [19]. Но для моделей малого размера результаты могут различаться. Мы решили протестировать эти гипотезы, проведя эксперимент. Вот такой результат мы получили, применяя LoRa к query и value для rank=1..256, alpha=1..256 на датасете mrpc, на сиде 42 по метрике f1 (таблица 1):

Таблица 1. LoRa rank, alpha, seed=42, dataset: MRPC, метрика: f1

	a1	a2	a4	a8	a16	a32	a64	a128	a256
r1	0.915	0.920	0.921	0.927	0.932	0.915	0.913	0.812	0.851
r2	0.914	0.917	0.911	0.913	0.920	0.918	0.921	0.913	0.812
r4	0.912	0.927	0.911	0.910	0.929	0.919	0.812	0.912	0.892
r8	0.911	0.913	0.915	0.919	0.926	0.939	0.923	0.931	0.834
r16	0.919	0.925	0.919	0.920	0.929	0.921	0.936	0.932	0.918
r32	0.918	0.918	0.923	0.920	0.924	0.930	0.928	0.812	0.812
r64	0.910	0.918	0.923	0.920	0.920	0.923	0.934	0.927	0.812
r128	0.917	0.926	0.919	0.925	0.928	0.920	0.934	0.923	0.835
r256	0.915	0.915	0.917	0.921	0.926	0.923	0.924	0.920	0.812

Аргументы к рис.1 (Приложение 4). Далее мы провели аналогичный эксперимент для сета 1 для rank=1..128, alpha=1..128 и получили аналогичные результаты (приложение 4). Также мы еще раз проверили на датасете rte ($lr=7e-4$) для rank=1..128, alpha=1..128 и получили похожие результаты. Довольно интересно, что при больших alpha (для alpha = 256) качество падает. Поэтому мы решили отдельно провести аналогичный эксперимент при меньшем lr на rte (с $7e-4$ до $2e-4$) и получили результаты лучше результаты на rte - (Приложение 5).

По данному эксперименту можно заключить выводы: при увеличении rank и alpha результаты становятся лучше, alpha немного сильнее влияет на результат (заметнее на датасете rte). Лучше использовать rank и alpha примерно похожие, тогда получается неплохой результат. Но что интересно,

что разные гипотезы по типу $\alpha=\text{rank}$, $\alpha=2\text{rank}$, не дают большого различия. Далее мы будем использовать $\text{rank}=16$, $\alpha=16$.

Далее мы решили проверить модули, к которым можно применять lora. В статье по lora авторы проверяли только для модулей из self-attention, оставляя [18]. В статье по Qlora авторы писали, что, применяя ко всем модулям результат получается лучше [22]. Но мы решили проверить более подробно, так как для малых моделей разница в скорости и нагрузке на GPU для разных модулей – ощутимая, мы проверили вместе с перебором модулей из MLP. В начале мы проверили эксперимент на трех сидах [42, 1, 2] для query, value, key, также включив, применение lora ко всему self-attention (exp-lora-q-v-k-attff), и ко всем слоям (exp-lora-all-layers) на для mrpc и на двух сидах для cb (таблица 2). Также более подробно (Приложение 6).

Таблица 2. LoRa модули MRPC и RTE

name	f1_avg	name	acc_avg	train_runtime
exp-lora-all-layers	0.9284	exp-rte-lora-intff	0.7978	657.792925
exp-lora-v-k	0.9262	exp-rte-lora-attff	0.7753	645.364200
exp-lora-v	0.9251	exp-rte-lora-attff-intff	0.7419	676.715375
exp-lora-q-v-k-attff	0.9248	exp-rte-lora	0.7238	624.096325
exp-lora-q-v	0.9240	exp-rte-lora-intff-outff	0.7157	709.735025
exp-lora-q-v-k	0.9229	exp-rte-lora-all-layers	0.6742	724.451150
exp-lora-q-k	0.9166	exp-rte-lora-outff-attff	0.6588	680.174050
exp-lora-q	0.9161	exp-rte-lora-outff	0.6552	663.565975
exp-lora-k	0.9141			

Посмотрев на результаты, можно увидеть, что применение ко всем слоям, действительно, показывает лучшие результаты. Но что более интересно, что применение к модулям: [query, value, key], [value, key], [query, value], [value] показывает почти такие же результаты. А что интересно, что сильнее всего на результат положительно влияет value матрица. Далее мы решили более подробно рассмотреть применение к остальным слоям для [query, value, key], [value, key], [query, value], [value] сид 42, $r=16$, $a=16$. Результат для mrpc и rte (таблица 1) Более подробно для rte (Приложение 7).

Про результат для mrpc было описано ранее. На rte – усреднение qv_k , qv , vk , v , где $attff$ – выходная матрица в self-attention, $intff$ – первый линейный слой MLP, $outff$ – второй линейный слой MLP. Также такой же эксперимент был проведен для mrpc на 2x T4 GPU (приложение 8). Из этих экспериментов можно сделать следующие выводы: добавление модулей положительно влияет на результат. Причем из экспериментов по rte и mrpc можно сделать вывод, что добавление модулей в $intff$ (первый линейный слой MLP) и value вектора дает наибольший прирост. Причем соотношение по времени $(rte_lora_intff) / (rte_lora) \approx 1.05$, а $(rte_lora_all_layers) / (rte_lora_intff) \approx 1.1$. Что значит, что v и $intff$ работает относительно быстро. Также стоит отметить, что $(lora_all_layers) / (lora_1_layer) \approx 1.25$, что значит, что добавление всех модулей может увеличить время работы примерно на 25%.

Также мы проверили зависимость скорости обучения от rank и alpha в lora и получили, что от alpha зависимости нет, а от r – не более 10% (Приложение 9).

(IA)³: для ia3 мы, как обычно, начали с $learning_rate$ и получили, что на промежутке: $[5e-4, 5e-2]$ лучше всего работает $5e-3$, поэтому решили использовать его в дальнейшем (Приложение 10).

Для ia3 мы тоже проверили модули. В начале мы провели эксперимент для query, key, value на датасете mrpc по сидам [42, 1] и получили, что лучшие

результаты показали: qvk, vk, qv (Приложение 11). Проведя аналогичный эксперимент на cb, мы получили: qv, vk, qvk. Далее мы взяли два лучших результата их mrpc и добавили к ним feedforward (Приложение 11). На mrpc лучшие результаты показали intff-outff, outff, attff-outff. Еще можно заметить, что все они показали результаты лучше, чем без добавления векторов, кроме query, value, key. Причем, т.к. в ia3 около 0.02% параметров от всей модели, то время обучения при подключении всех слоев практически не изменилось: не более 10%. Также проверив для cb, мы получили, что intff-outff и attff-outff показали наилучшие результаты.

AdaLoRa: начав с learning_rate на [1e-5, 5e-2] мы получили хороший результат на [1e-3, 5e-3] и решили продолжать с 1e-3, 2e-3 (Приложение 12). Далее мы решили рассмотреть модули и начали с query, key, value. Мы начали с датасета MRPC. После проведения эксперимента на сидах [42, 1] мы получили, что лучшие результаты показали: qv, v, vk, qvk (Приложение 13). Дополнительно можно заметить, что value модуль сильнее всего влияет на результат. Мы провели такой же результат на rte и получили похожие результаты: qvk, v, vk, qv показали лучшую точность (Приложение 14). Поэтому мы решили взять их и проверить дополнительные модули вместе с ними. Сделав это на датасете mrpc, мы получили, что лучшие результаты были на all-layers и intff. Прделаав тоже самое на rte получили: all-layers и intff-outff. Отсюда можно сделать вывод, что all-layers явно дает лучший результат, что ожидаемо для adalora. Также, как и в lora value и intff матрицы дают наибольший вклад в общий результат. Сравнив их по скорости, мы получили, что если применить адалору ко всем слоям, то она будет работать примерно в 1.5 раза дольше. Т. е. добавляя новый слой в адалору мы сильнее замедляемся, чем при добавлении в лору.

Сравнение: для сравнения были использованы 4 датасета на сидах, которые до этого модели не видели. Первые 3 – по 3 сида: [10, 11, 12]. Boolq – 1 сид.

Так как на mrpc есть результаты тестовой выборки в открытом доступе, то мы обучили 10 эпох на mrpc, взяли максимум по валидационной выборке и посмотрели результаты на тесте (таблица 3):

Таблица 3. Сравнение на MRPC, усреднение по 3 сидам

name	test_acc	test_f1	eval_acc	eval_f1	time	params	dataset
adalora-all-layers	0.888	0.918	0.900	0.927	675.989	2.03	mrpc
adalora-v-intff	0.878	0.910	0.897	0.926	521.976	1.08	mrpc
lora-v-intff	0.873	0.907	0.889	0.921	469.417	1.29	mrpc
ia3	0.869	0.904	0.881	0.915	457.020	0.53	mrpc
fullft	0.853	0.893	0.877	0.911	688.971	100.00	mrpc
lora-all-layers	0.852	0.894	0.869	0.908	569.031	2.54	mrpc

Мы получили, что лучше всех себя показывает adalora, немного хуже lora и послабее: ia3 и fullft. Больше всего времени на обучение заняла полная разморозка и adalora на всех слоях. Меньше всего – ia3. Также хочется отметить, что adalora-v-intff и lora-v-intff неплохо себя показали, относительно затраченного времени. Также мы провели аналогичные замеры на других датасетах (Приложение 15) и получили аналогичные результаты: лучше всего себя показывают адалора, примененная ко всем слоям, адалора, примененная к value и первому слою из MLP и лора, примененная к value и первому слою из MLP. Также хочется отметить, что lora-v-intff в среднем обучается на 30% быстрее.

Далее мы проверили сравнение по нагрузке на GPU (Приложение 16). Мы получили: lora и ia3 расходуют около 2/3 GPU от fullft, (авторы статьи по Lora заявляли около 1/3 на GPT-3 175B, но у нас модель в 1000 раз меньше, поэтому может быть не настолько большая разница). Если применять к большему количеству слоев, то расход значительно увеличивается, γ – не так сильно влияет, как количество target модулей. IA3 дает наименьшую нагрузку на GPU, но почти не отличается от lora. Для определения погрешности экспериментов воспользуемся стандартной ошибкой среднего (несмещенная выборочная дисперсия / корень из n (количество экспериментов) для датасета mrpc на тесте и метрикой ассигасу. Получаем для тестовой выборки в 1.73 тыс. * 3 сйда. объектов: fullft: 0.005, lora-v-intff: 0.0046, adalora-all-layers: 0.0044, т. е. немного менее 0.5% по ассигасу. Также можно рассмотреть t-критерий для различия средних. Для fullft и lora-v-intff на mrpc получаем, что различия статистически значимы (значение: 2.9, критическое: 1.97). Проверив на всех датасетах, мы получили, что различие между лорой и методом полной разморозки – статистически значимо, а результаты для лоры и адалоры: - тоже значимы (значение: 2.3, критическое: 1.97). Среднее значение стандартной ошибки для датасетов: mrpc: 0.0047, cb: 0.016, rte: 0.014, boolq: 0.0071. В итоге получаем: lora > fullft (все датасеты) статистически значимо, adalora > lora (mrpc, rte) статистически значимо, lora > adalora (boolq) – статистически не значимо.

2.2 Выводы

При изучении результатов исследования, можно сделать следующие выводы:

1. Методы PEFT в среднем обучаются быстрее, чем метод полной разморозки, до 1.5 раз быстрее на LLM малого размера.
2. Методы PEFT расходуют в среднем меньше памяти, чем метод полной разморозки, примерно 2/3 от метода full fine-tuning.
3. Лучшие rank и alpha в методе lora примерно от 8 до 64, примерно равные, но при меньшем r результаты относительно неплохие. Гипотезы по типу $\alpha=2r$ и т. д. - не подтвердились.
4. Чем больше модулей для применения в lora и adalora, тем лучше себя показывает модель, также довольно важно добавлять модули вне query, value, key. Но, что интересно, среди query, value, key: value дает наибольший прирост, а среди остальных: первый линейный слой в MLP. Комбинацию value + intff мы нашли самой лучшей из вариаций для двух модулей, во многих случаях показывает результат на уровне all-layers.
5. (IA)³ – самый быстрый для дообучения, хотя больше прироста в экономии GPU по сравнению с lora не было замечено. Также, за счет того, что в нем меньше всего параметров: около 0.02% получается, что в нем добавление модулей не настолько сильно ухудшает скорость обучения, но влияет на GPU. Среди модулей сильнее всего на результат работы ia3 влияют v, k и outff. Но результаты ia3 на малых LLM получаются немного хуже, чем lora.
6. Добавление модулей сильнее влияет на нагрузку на GPU и скорость обучения, чем увеличение ранга. Добавление модулей уменьшает скорость обучения lora до 25%, adalora – до 50%, для ia3 – 10%. А для

rank – не более 10%. Для нагрузки на GPU результаты аналогичные: добавление всех модулей уменьшает примерно на треть, а rank – около 10%.

7. Лучше всего по качеству себя показывают adalora и lora. Различие между лорой и полной разморозкой – статистически значимо на всех датасетах. Мы нашли, что в среднем лучший результат дает adalora all-layers.

Заключение

Основная проблема, на изучение которой была направлена исследовательская работа - анализировать методы дообучения, сравнить их между собой и выявить лучшие по качеству, времени, памяти.

Для выполнения поставленной цели была проведена следующая работа: изучена и проанализирована литература по проблеме исследования; охарактеризованы основы языковых моделей малого размера и методы их дообучения; проведены эксперименты по дообучению с использованием разных методов и по подбору параметров различных методов.

В теоретической части работы рассматривалось структура языковых моделей малого размера; модели, предшествующие большим языковым моделями, такие как, классические нейронные модели; рассмотрены языковые модели: рекуррентные сети, LSTM, Seq2seq; основные понятия машинного обучения в области NLP; стандартные типы задач; популярные датасеты, используемые для обучения и тестирования моделей; архитектура трансформера и современные большие языковые модели малого размера, основанные на ней, такие как BERT и RoBERTa; основные методы дообучения больших языковых моделей, как методы, которые оставляют дообучение всей модели, классические методы: LoRa, AdaLoRa.

В практической части работы были проведены эксперименты по дообучению RoBERTa с использованием 4 методов: FullFT, LoRa, (IA)3, AdaLoRa, вместе с последующими измерениями и сравнениями по качеству, времени и памяти. Результаты показали, что PEFT модели обучают до 1.5 раз быстрее и расходуют на 1/3 памяти меньше, чем метод полной разморозки, показывая результаты не хуже, а часто – и лучше. Лучшие результаты показали LoRa и AdaLoRa.

В ходе исследования нами выявлено, что адаптеры и современные методы дообучения дают результат, не хуже, а часто и лучше, чем классическое дообучение методом полной разморозки, используя при этом меньше ресурсов, такие как память и время.

Анализ полученных результатов позволил выявить лучшие методы для дообучения языковых моделей малого размера. Различия между PEFT методами (LoRa и AdaLoRa) и методом полной разморозки – статистически значимы.

На основании данной работы нами предложены следующие рекомендации: использовать современные способы дообучения по типу: Parameter-Efficient Fine-Tuning методов.

Теоретическая и практическая значимость результатов состоит в предоставлении данных для применения методов дообучения предобученных моделей исходя из конкретных ограничений ресурсов, таких как видеопамять и время, в настройке параметров, а также в предоставлении готового кода для дообучения моделей, используя современную библиотеку `huggingface`.

Список использованных источников

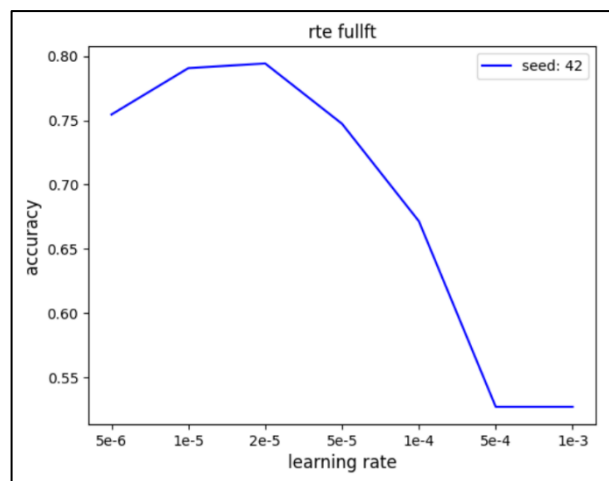
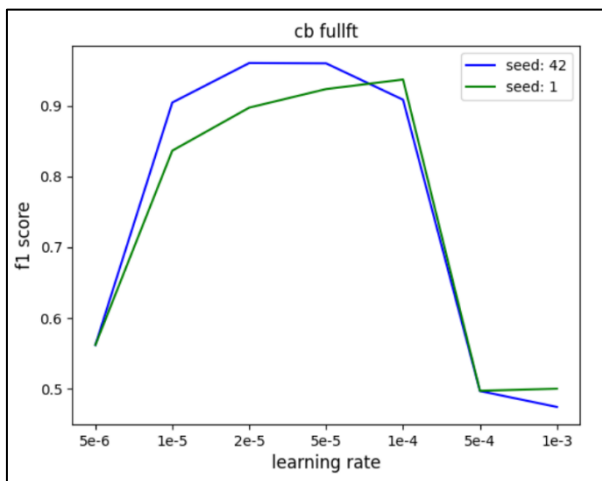
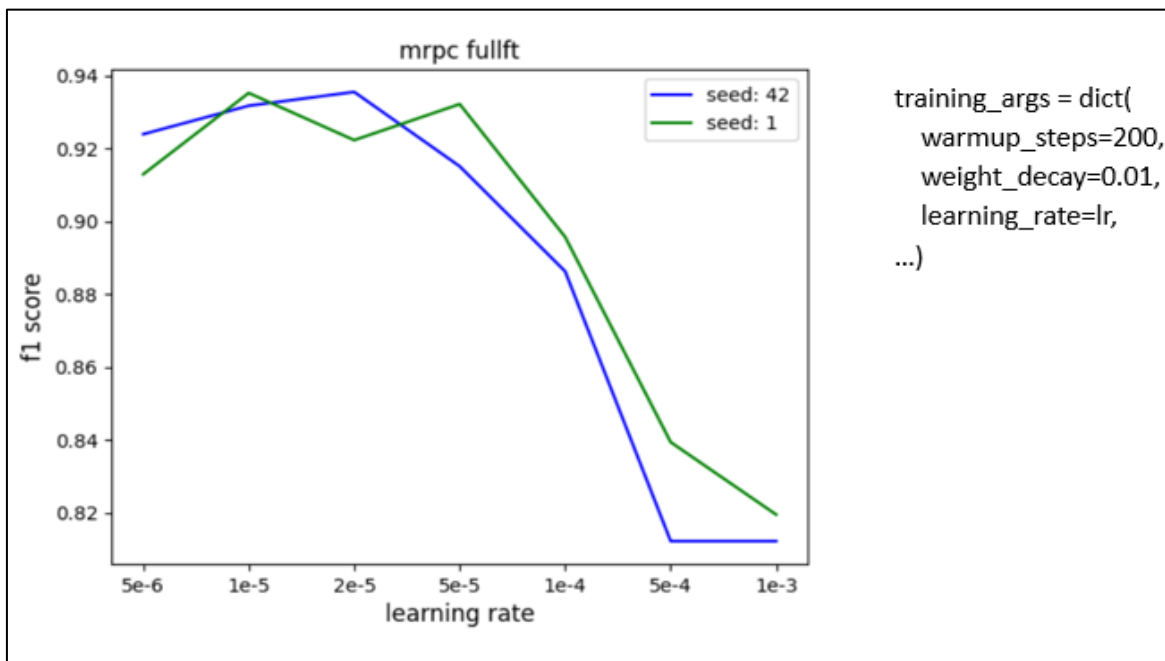
- [1] Осипов Ю.С., гл. ред. Большая российская энциклопедия — 2004-2017. — Т. 35. Электрон. ресурс. // URL:<https://bigenc.ru/c/neironnye-seti-e734b3> (дата обращения: 05.06.2024)
- [2] Розенблатт Ф.; пер. с англ. Алтаева В.Я. [и др.] Принципы нейродинамики. Перцептроны и теория механизмов мозга // Мир, 2004-2017. С. 480 Электрон. ресурс. URL:<https://bigenc.ru/b/printsipy-neirodinamiki-pe-85ccde?page=1> (дата обращения: 05.06.2024)
- [3] Сущеня Р.В., Кокаев А.Э. Нейронные сети и их классификация. Основные виды нейронных сетей // Международный научный журнал: «Вестник Науки». – 2023. Т. 1. – С. 186-188. // Электрон. ресурс. URL:<https://cyberleninka.ru/article/n/neyronnye-seti-i-ih-klassifikatsiya-osnovnye-vidy-neyronnyh-setey> (дата обращения: 05.06.2024)
- [4] Ксенофонов В.В. Нейронные сети // Проблемы науки. — 2020. — С. 28–29. // Электрон. ресурс. URL:<https://cyberleninka.ru/article/n/neyronnye-seti-1/viewer> (дата обращения: 05.06.2024)
- [5] Hadi M. U., Al-Tashi Q., Qureshi R., Shah A., Muneer A., Irfan M., Zafar A., Shaikh M. B., Akhtar N., Wu J., Mirjalili S. A Survey on Large Language Models: Applications, Challenges, Limitations, and Practical Usage. - 2023. Pp.1-30.
URL:https://www.researchgate.net/publication/372258530_Large_Language_Models_A_Comprehensive_Survey_of_its_Applications_Challenges_Limitations_and_Future_Prospects (дата обращения: 05.06.2024)
- [6] Khurana D., Koli A., Khatter K. Natural language processing: state of the art, current trends and challenges – 2022. -Vol. 82 - Pp. 3713–3744. URL: <https://link.springer.com/article/10.1007/s11042-022-13428-4> (дата обращения: 11.06.2024)

- [7] Козлова Н.В. Лингвистические корпуса: определение основных понятий и типология // Вестник Новосибирского государственного университета. Серия: Лингвистика и межкультурная коммуникация. Т. 11 - 2013 С. 79-88 Электрон. ресурс. URL: <https://korpusa-opredelenie-osnovnyh-ponyatiy-i-tipologiya> (дата обращения 11.06.2024)
- [8] *Khurana D., Koli A., Khatter K.* Natural language processing: state of the art, current trends and challenges – 2023. - Vol. 82 -Pp. 3713–3744 URL: <https://link.springer.com/article/10.1007/s11042-022-13428-4> (дата обращения 11.06.2024)
- [9] *Wang A., Singh A., Michael J., Hill F., Levy O., Bowman S.R.* GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding - 2019. Pp. 20 URL: <https://openreview.net/pdf?id=rJ4km2R5t7> (дата обращения 11.06.2024)
- [10] *Wang A., Pruksachatkun Y., Nangia N., Singh A., Michael J., Hill F., Levy O., Bowman S. R.* SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems -2019. Pp. 30 URL: <https://w4ngatang.github.io/static/papers/superglue.pdf> (дата обращения 11.06.2024)
- [11] *Sutskever I., Vinyals O., Le Q. V.* Sequence to Sequence Learning with Neural Networks – 2014. Pp. 1-9. URL: <https://arxiv.org/pdf/1409.3215> (дата обращения: 13.06.2024)
- [12] *Bahdanau D., Cho K., Bengio Y.* Neural Machine Translation by Jointly Learning to Align and Translate – 2014-2016. URL: <https://arxiv.org/abs/1409.0473> (дата обращения: 13.06.2024)
- [13] *Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser L., Polosukhin I.* Attention Is All You Need – 2017-2023. URL: <https://arxiv.org/abs/1706.03762> (дата обращения: 14.06.2024)
- [14] *Devlin J., Chang M.-W., Lee K., Toutanova K.* BERT: Pre-training of Deep

- Bidirectional Transformers for Language Understanding – 2018-2019. URL: <https://arxiv.org/abs/1810.04805> (дата обращения: 14.06.2024)
- [15] *Liu Y., Ott M., Goyal N., Du J., Joshi M., Chen D., Levy O., Lewis M., Zettlemoyer L., Stoyanov V.* RoBERTa: A Robustly Optimized BERT Pretraining Approach – 2019. URL: <https://arxiv.org/abs/1907.11692> (дата обращения: 14.06.2024)
- [16] *Jahan M.S., Oussalah M., Beddia D.R., Mim J.K., Arhab N.* A Comprehensive Study on NLP Data Augmentation for Hate Speech Detection: Legacy Methods, BERT, and LLMs – 2024. Pp. 31. URL: <https://arxiv.org/abs/2404.00303> (дата обращения 15.06.2024)
- [17] *Lialin V., Deshpande V., Rumshisky A.* Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning – 2023. URL: <https://arxiv.org/abs/2303.15647> (дата обращения 15.06.2024)
- [18] *Hu E.J., Shen Y., Wallis P., Allen-Zhu Z., Li Y., Wang S., Wang L., Chen W.* LoRA: Low-Rank Adaptation of Large Language Models - 2023. URL: <https://arxiv.org/abs/2106.09685> (дата обращения 15.06.2024)
- [19] *Aghajanyan A., Zettlemoyer L., Gupta S.* Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning – 2020. URL: <https://arxiv.org/abs/2012.13255> (дата обращения 15.06.2024)
- [20] *Zhang Q., Chen M., Bukharin A., Karampatziakis N., He P., Cheng Y., Chen W., Zhao T.* AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning – 2023. URL: <https://arxiv.org/abs/2303.10512>
- [21] Электронный ресурс // Sebastian Raschka “Finetuning LLMs with LoRA and QLoRA: Insights from Hundreds of Experiments” – Posted on October 12 2023. - URL: <https://lightning.ai/pages/community/lora-insights/>
- [22] *Dettmers T., Pagnoni A., Holtzman A., Zettlemoyer L.* QLoRA: Efficient Finetuning of Quantized LLMs - 2023. URL: <https://arxiv.org/abs/2305.14314> (дата обращения 15.06.2024)

ПРИЛОЖЕНИЕ 1

learning_rate для полной разморозки



ПРИЛОЖЕНИЕ 2

weight_decay и warmup_steps для mrpc и cb

датасет: mrpc
сид: 42
метрика: f1

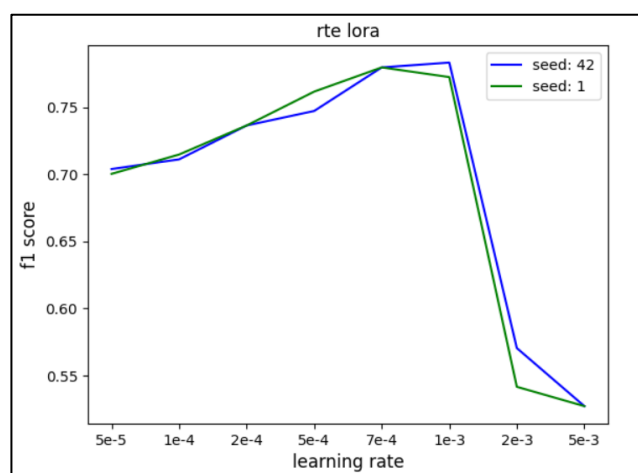
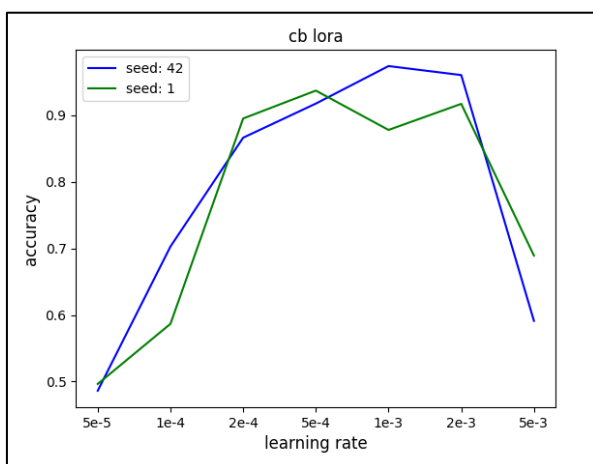
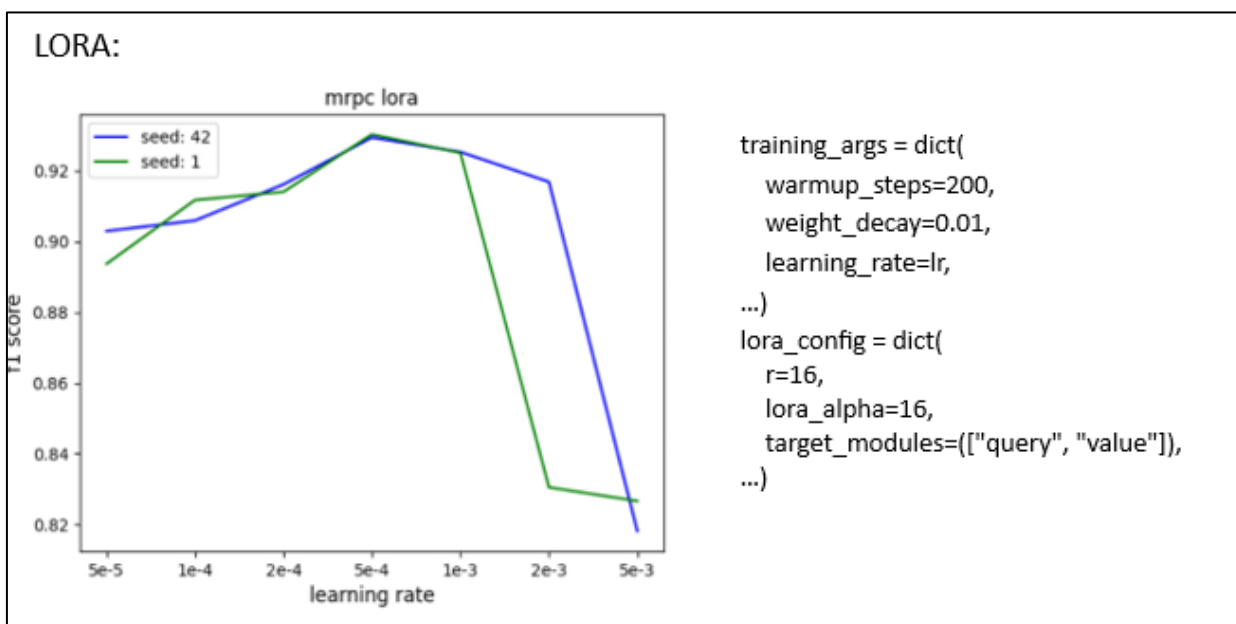
	0	200	500
0	0.9269	0.9291	0.935
0.01	0.9324	0.9355	0.9338
0.05	0.9324	0.9231	0.9268
0.1	0.9312	0.9261	0.9315
0.2	0.9288	0.9279	0.9284

```
training_args = dict(
    warmup_steps=ws,
    weight_decay=wd,
    learning_rate=2e-5,
    ...)
```

	0	20	50	100
0	0.9602	0.9044	0.9736	0.9736
0.01	0.9602	0.9102	0.9602	0.9369
0.05	0.9503	0.9044	0.8969	0.9736
0.1	0.9467	0.8647	0.8789	0.9607
0.2	0.9467	0.8909	0.933	0.9605

ПРИЛОЖЕНИЕ 3

learning_rate lora



ПРИЛОЖЕНИЕ 4

lora rank и alpha датасет mrpc

rank, alpha:

датасет: mrpc

сиды: 42

метрика: f1

```
training_args = dict(  
    warmup_steps=200,  
    weight_decay=0.01,  
    learning_rate=5e-4,  
    ...)
```

```
lora_config = dict(  
    r=r,  
    lora_alpha=a,  
    target_modules=(["query", "value"]),  
    ...)
```

	a1	a2	a4	a8	a16	a32	a64	a128
r1	0.912	0.911	0.915	0.915	0.923	0.923	0.917	0.815
r2	0.914	0.913	0.913	0.916	0.918	0.920	0.911	0.842
r4	0.920	0.920	0.922	0.925	0.916	0.927	0.928	0.929
r8	0.912	0.922	0.918	0.917	0.927	0.923	0.921	0.924
r16	0.915	0.921	0.920	0.923	0.930	0.925	0.932	0.921
r32	0.923	0.918	0.924	0.923	0.930	0.913	0.933	0.812
r64	0.910	0.919	0.928	0.925	0.928	0.928	0.928	0.917
r128	0.912	0.919	0.922	0.930	0.930	0.916	0.932	0.928

ПРИЛОЖЕНИЕ 5

lora rank и alpha датасет rte

	a1	a2	a4	a8	a16	a32	a64	a128
r1	0.729	0.740	0.762	0.736	0.780	0.765	0.527	0.585
r2	0.747	0.747	0.740	0.794	0.758	0.765	0.650	0.527
r4	0.751	0.736	0.776	0.769	0.791	0.787	0.588	0.664
r8	0.729	0.740	0.747	0.776	0.773	0.769	0.542	0.527
r16	0.729	0.751	0.769	0.780	0.780	0.744	0.776	0.527
r32	0.751	0.758	0.755	0.780	0.762	0.542	0.653	0.527
r64	0.733	0.758	0.769	0.809	0.776	0.542	0.755	0.527
r128	0.733	0.726	0.736	0.798	0.769	0.762	0.740	0.527

	a128
r1	0.726
r2	0.780
r4	0.527
r8	0.527
r16	0.747
r32	0.527
r64	0.809
r128	0.805

ПРИЛОЖЕНИЕ 6

LoRa target modules. Датасеты: mrpc, cb

name	f1_avg	
exp-lora-all-layers	0.9284	<pre> training_args = dict(warmup_steps=200, weight_decay=0.01, learning_rate=5e-4, ...) lora_config = dict(r=16, lora_alpha=16, target_modules=(modules), ...)</pre>
exp-lora-v-k	0.9262	
exp-lora-v	0.9251	
exp-lora-q-v-k-attff	0.9248	
exp-lora-q-q-v	0.9240	
exp-lora-q-v-k	0.9229	
exp-lora-q-k	0.9166	
exp-lora-q	0.9161	
exp-lora-k	0.9141	

name	f1_avg
exp-lora-all-layers	0.9934
exp-lora-v	0.9686
exp-lora-q-v-k	0.9605
exp-lora-q-v-k-attff	0.9519
exp-lora-k	0.9406
exp-lora-q-q-v	0.9272
exp-lora-v-k	0.9105
exp-lora-q-k	0.8870
exp-lora-q	0.8715

ПРИЛОЖЕНИЕ 7

MLP модули lora rte

name	acc_avg	train_runtime
exp-rte-lora-qvk-intff	0.8014	694.5637
exp-rte-lora-qvk-all-layers	0.7942	761.0796
exp-rte-lora-qvk-intff-outff	0.7870	732.8347
exp-rte-lora-qvk-outff-attff	0.7834	717.8495
exp-rte-lora-qvk-outff	0.7834	681.0972
exp-rte-lora-qvk-attff	0.7798	670.2696
exp-rte-lora-qvk	0.7653	645.9130
exp-rte-lora-qvk-attff-intff	0.5812	712.1251

name	acc_avg	train_runtime
exp-rte-lora-qv-attff-intff	0.8051	694.8088
exp-rte-lora-qv-intff	0.8014	673.2754
exp-rte-lora-qv-all-layers	0.7870	744.2766
exp-rte-lora-qv-attff	0.7798	648.2000
exp-rte-lora-qv-intff-outff	0.7726	709.9792
exp-rte-lora-qv	0.5740	624.2966
exp-rte-lora-qv-outff	0.5415	663.8426
exp-rte-lora-qv-outff-attff	0.5271	697.3113

name	acc_avg	train_runtime
exp-rte-lora-vk-intff	0.7942	629.5147
exp-rte-lora-vk-attff-intff	0.7870	646.6378
exp-rte-lora-vk-outff-attff	0.7798	651.5447
exp-rte-lora-vk-attff	0.7762	653.0115
exp-rte-lora-vk-intff-outff	0.7762	712.4848
exp-rte-lora-vk-outff	0.7690	668.9776
exp-rte-lora-vk	0.7653	633.0500
exp-rte-lora-vk-all-layers	0.5271	693.9007

name	acc_avg	train_runtime
exp-rte-lora-v-intff	0.7942	633.8179
exp-rte-lora-v-attff-intff	0.7942	653.2898
exp-rte-lora-v	0.7906	593.1257
exp-rte-lora-v-attff	0.7653	609.9757
exp-rte-lora-v-all-layers	0.5884	698.5477
exp-rte-lora-v-outff-attff	0.5451	653.9907
exp-rte-lora-v-outff	0.5271	640.3465
exp-rte-lora-v-intff-outff	0.5271	683.6414

ПРИЛОЖЕНИЕ 8

MLP модули lora mrpc

name	f1_avg	train_runtime
exp-mrpc-lora-attff-intff	0.9270	447.862125
exp-mrpc-lora-intff	0.9251	425.798075
exp-mrpc-lora-all-layers	0.9242	482.832675
exp-mrpc-lora-outff-attff	0.9232	443.759525
exp-mrpc-lora-attff	0.9228	419.420100
exp-mrpc-lora	0.9211	411.216075

name	f1_avg	train_runtime
exp-mrpc-lora-qvk-outff-attff	0.9282	468.7719
exp-mrpc-lora-qvk-attff-intff	0.9269	470.5799
exp-mrpc-lora-qvk-intff	0.9260	442.0136
exp-mrpc-lora-qvk	0.9250	431.7520
exp-mrpc-lora-qvk-all-layers	0.9244	507.1300
exp-mrpc-lora-qvk-attff	0.9223	450.4081

name	f1_avg	train_runtime
exp-mrpc-lora-qv-attff-intff	0.9345	445.1344
exp-mrpc-lora-qv-all-layers	0.9231	487.3390
exp-mrpc-lora-qv-intff	0.9217	417.4430
exp-mrpc-lora-qv-attff	0.9215	404.7754
exp-mrpc-lora-qv-outff-attff	0.9184	436.7127
exp-mrpc-lora-qv	0.9123	404.4049

name	f1_avg	train_runtime
exp-mrpc-lora-vk-intff	0.9287	429.9423
exp-mrpc-lora-vk-all-layers	0.9236	483.0979
exp-mrpc-lora-vk-outff-attff	0.9234	444.9172
exp-mrpc-lora-vk	0.9231	420.6702
exp-mrpc-lora-vk-attff	0.9231	419.2640
exp-mrpc-lora-vk-attff-intff	0.9228	446.8984

name	f1_avg	train_runtime
exp-mrpc-lora-v-all-layers	0.9257	453.7638
exp-mrpc-lora-v-attff	0.9245	403.2329
exp-mrpc-lora-v	0.9242	388.0372
exp-mrpc-lora-v-intff	0.9241	413.7934
exp-mrpc-lora-v-attff-intff	0.9236	428.8358
exp-mrpc-lora-v-outff-attff	0.9228	424.6363

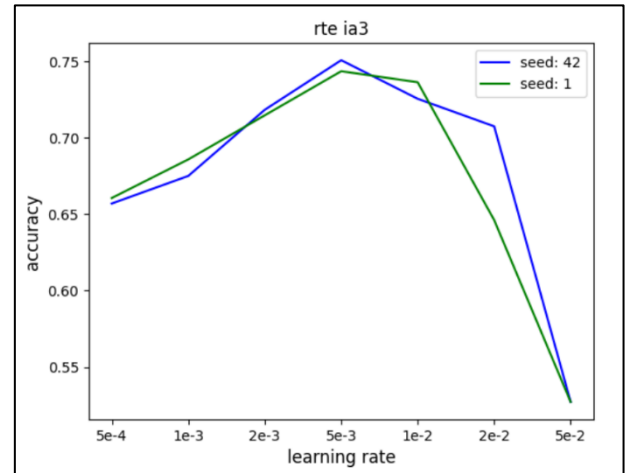
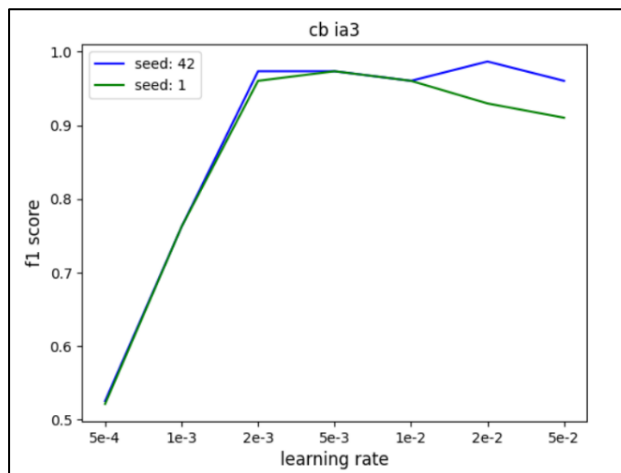
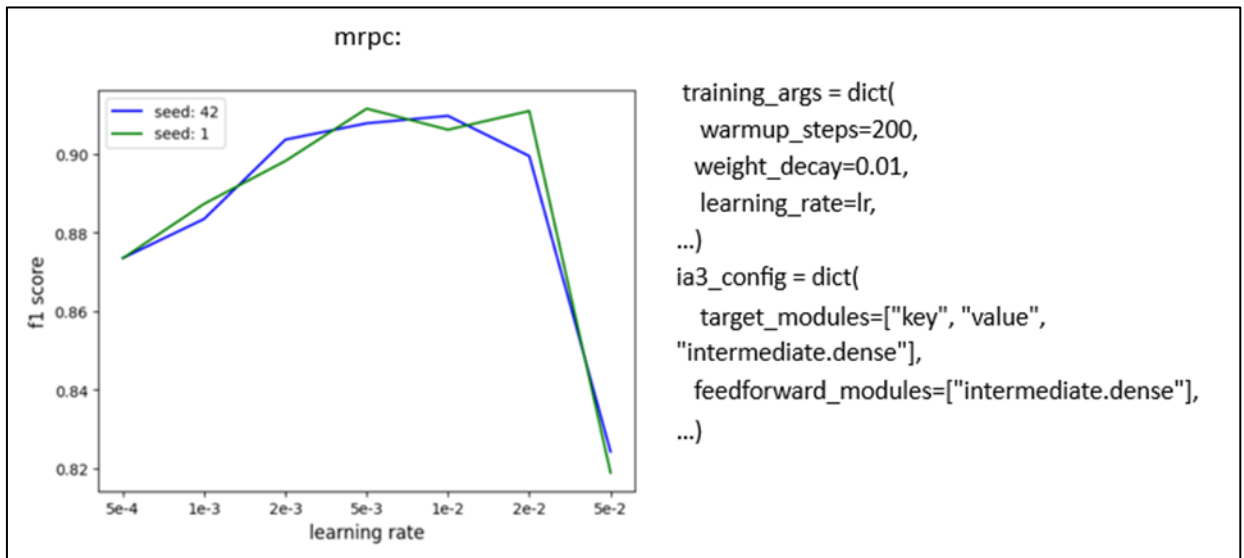
ПРИЛОЖЕНИЕ 9

log₁₀ скорость обучения от rank и alpha

	a1	a2	a4	a8	a16	a32	a64	a128
r1	655.029	655.275	684.476	685.01	686.821	687.088	683.934	660.625
r2	672.488	672.534	673.105	673.544	673.747	673.516	671.431	671.482
r4	674.602	674.485	674.203	674.145	674.234	675.114	672.01	672.133
r8	675.021	648.377	656.322	708.167	711.416	711.563	709.018	707.748
r16	693.912	693.272	693.4	695.569	694.579	694.925	695.958	693.538
r32	698.237	698.682	709.956	699.296	699.502	695.474	678.235	682.964
r64	695.868	696.153	696.414	696.625	696.326	694.321	697.211	694.601
r128	720.227	721.399	720.909	720.36	720.714	721.34	720.974	725.788

ПРИЛОЖЕНИЕ 10

ia3 learning_rate



ПРИЛОЖЕНИЕ 11

Модули ia3

name	f1_avg	
exp-mrpc-ia3-q-v-k	0.906271	<pre> training_args = dict(warmup_steps=200, weight_decay=0.01, learning_rate=5e-3, ...) ia3_config = dict(target_modules=modules, # feedforward_modules=None, ...)</pre>
exp-mrpc-ia3-v-k	0.900172	
exp-mrpc-ia3-q-v	0.899269	
exp-mrpc-ia3-q-k	0.893514	
exp-mrpc-ia3-v	0.890292	
exp-mrpc-ia3-q	0.88813	
exp-mrpc-ia3-k	0.887002	

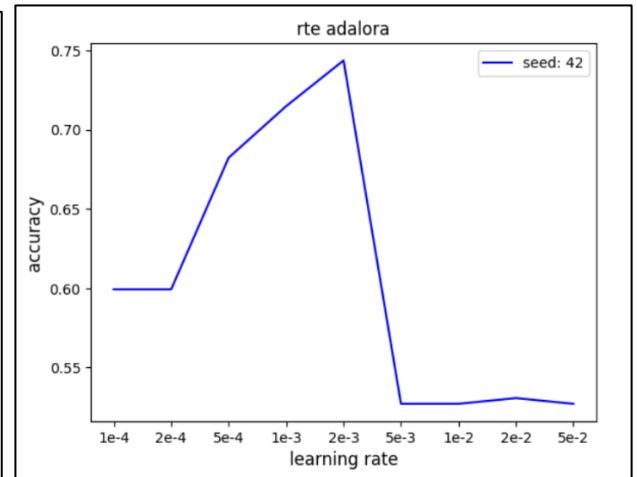
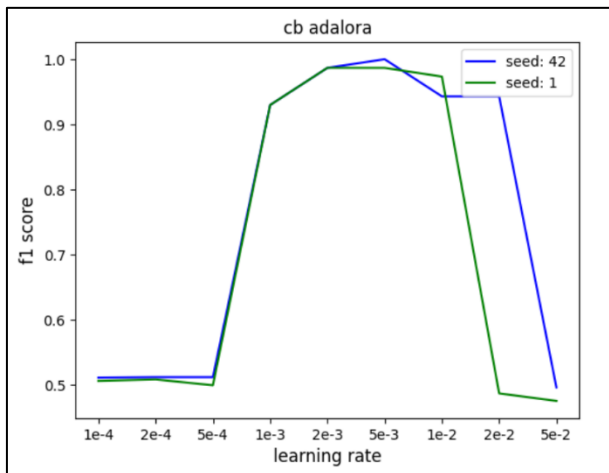
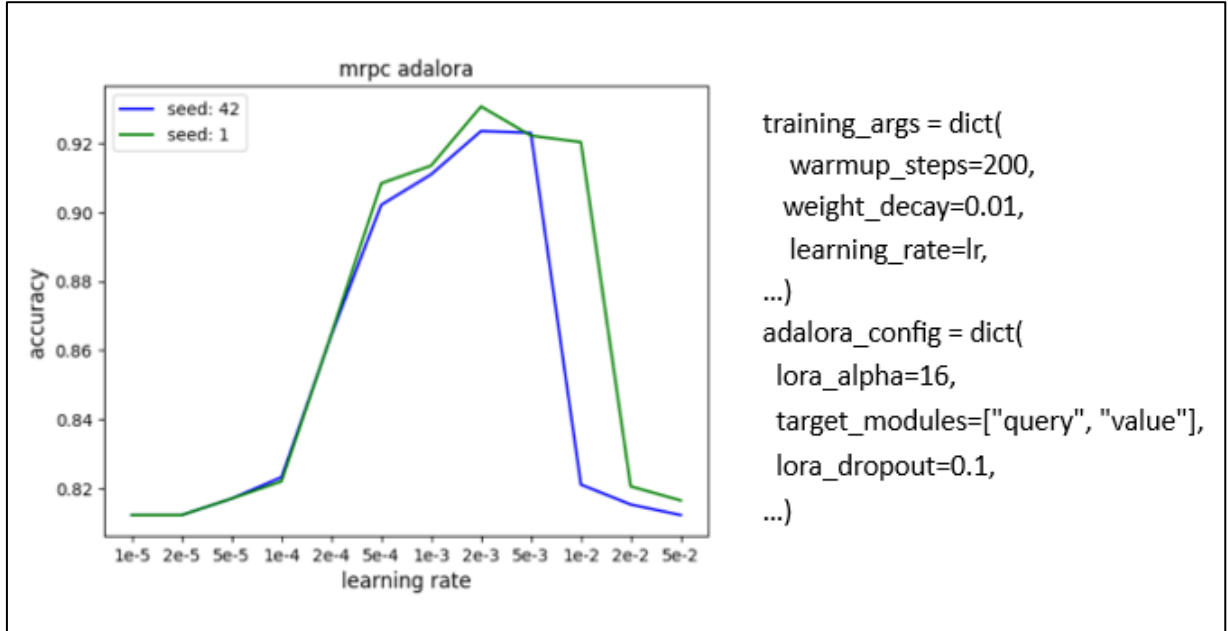
name	f1_avg
exp-cb-ia3-q-v	0.947531
exp-cb-ia3-v-k	0.940883
exp-cb-ia3-q-v-k	0.93454
exp-cb-ia3-k	0.921538
exp-cb-ia3-q-k	0.921538
exp-cb-ia3-q	0.921417
exp-cb-ia3-v	0.842856

name	f1_avg	train_runtime
exp-mrpc-ia3-qvk-intff-outff	0.9178	528.33275
exp-mrpc-ia3-qvk-outff-attff	0.9161	530.02735
exp-mrpc-ia3-qvk-outff	0.9153	518.39560
exp-mrpc-ia3-qvk-attff-intff	0.9148	541.51890
exp-mrpc-ia3-qvk-all-layers	0.9138	560.16890
exp-mrpc-ia3-qvk-attff	0.9083	527.40630
exp-mrpc-ia3-qvk-intff	0.9079	534.17805

name	f1_avg	train_runtime
exp-mrpc-ia3-qvk-outff-attff	0.9160	523.13235
exp-mrpc-ia3-qvk-outff	0.9142	516.32650
exp-mrpc-ia3-qvk-all-layers	0.9124	529.89545
exp-mrpc-ia3-qvk-intff-outff	0.9121	522.66040
exp-mrpc-ia3-qvk-attff-intff	0.9103	507.36575
exp-mrpc-ia3-qvk-intff	0.9097	501.97505
exp-mrpc-ia3-qvk-attff	0.9041	504.86550

ПРИЛОЖЕНИЕ 12

AdaLoRa: learning_rate



ПРИЛОЖЕНИЕ 13

AdaLoRa: модули mrpc

name	f1_avg
exp-adalora-qv	0.9272
exp-adalora-v	0.9227
exp-adalora-vk	0.9227
exp-adalora-qvk	0.9187
exp-adalora-k	0.9124
exp-adalora-q	0.9102
exp-adalora-qk	0.9089

name	f1_avg	train_runtime
exp-mrpc-adalora-all-layers	0.9294	760.205950
exp-mrpc-adalora-intff	0.9272	661.223425
exp-mrpc-adalora-attff-intff	0.9257	700.916450
exp-mrpc-adalora-outff	0.9257	641.200950
exp-mrpc-adalora-outff-attff	0.9247	677.959875
exp-mrpc-adalora-attff	0.9230	623.134675
exp-mrpc-adalora-intff-outff	0.9227	718.317700
exp-mrpc-adalora	0.9211	586.317975

name	f1_avg	train_runtime
exp-mrpc-adalora-qvk-all-layers	0.9350	823.0525
exp-mrpc-adalora-qvk-intff-outff	0.9297	752.3038
exp-mrpc-adalora-qvk-attff-intff	0.9283	757.5587
exp-mrpc-adalora-qvk-attff	0.9247	667.3326
exp-mrpc-adalora-qvk-intff	0.9234	713.4449
exp-mrpc-adalora-qvk-outff	0.9225	677.8490
exp-mrpc-adalora-qvk-outff-attff	0.9223	736.8732
exp-mrpc-adalora-qvk	0.9177	629.6210

name	f1_avg	train_runtime
exp-mrpc-adalora-qv-intff	0.9354	664.8848
exp-mrpc-adalora-qv-all-layers	0.9284	771.4681
exp-mrpc-adalora-qv-attff-intff	0.9271	704.0408
exp-mrpc-adalora-qv-outff-attff	0.9269	684.9396
exp-mrpc-adalora-qv-attff	0.9225	632.5894
exp-mrpc-adalora-qv	0.9204	596.1022
exp-mrpc-adalora-qv-outff	0.9201	639.5716
exp-mrpc-adalora-qv-intff-outff	0.9194	718.9939

name	f1_avg	train_runtime
exp-mrpc-adalora-vk-outff	0.9329	637.9410
exp-mrpc-adalora-vk-outff-attff	0.9302	662.9412
exp-mrpc-adalora-vk-all-layers	0.9284	741.1024
exp-mrpc-adalora-vk-attff-intff	0.9255	694.9190
exp-mrpc-adalora-vk	0.9252	584.8377
exp-mrpc-adalora-vk-intff	0.9239	655.0903
exp-mrpc-adalora-vk-attff	0.9225	622.7624
exp-mrpc-adalora-vk-intff-outff	0.9161	718.1837

name	f1_avg	train_runtime
exp-mrpc-adalora-v-outff	0.9271	609.4422
exp-mrpc-adalora-v-intff	0.9260	611.4737
exp-mrpc-adalora-v-all-layers	0.9258	705.2008
exp-mrpc-adalora-v-intff-outff	0.9255	683.7894
exp-mrpc-adalora-v-attff	0.9222	569.8543
exp-mrpc-adalora-v-attff-intff	0.9220	647.1473
exp-mrpc-adalora-v	0.9212	534.7110
exp-mrpc-adalora-v-outff-attff	0.9195	627.0855

ПРИЛОЖЕНИЕ 14

AdaLoRa: модули rte

name	acc_avg
exp-rte-adalora-qvk	0.7906
exp-rte-adalora-v	0.7545
exp-rte-adalora-vk	0.7545
exp-rte-adalora-qv	0.7437
exp-rte-adalora-qk	0.7401
exp-rte-adalora-k	0.6931
exp-rte-adalora-q	0.6751

name	acc_avg	train_runtime
exp-rte-adalora-all-layers	0.8159	803.449350
exp-rte-adalora-intff-outff	0.8123	791.085375
exp-rte-adalora-attff-intff	0.8114	741.846550
exp-rte-adalora-outff-attff	0.8069	719.563025
exp-rte-adalora-intff	0.8005	718.793675
exp-rte-adalora-outff	0.7987	705.982325
exp-rte-adalora-attff	0.7717	660.692150
exp-rte-adalora	0.7599	629.779850

name	acc_avg	train_runtime
exp-rte-adalora-qvk-intff-outff	0.8267	820.3635
exp-rte-adalora-qvk-all-layers	0.8159	845.8690
exp-rte-adalora-qvk-intff	0.8123	758.0581
exp-rte-adalora-qvk-attff-intff	0.8087	788.2997
exp-rte-adalora-qvk-outff-attff	0.7942	767.1739
exp-rte-adalora-qvk-outff	0.7942	739.0286
exp-rte-adalora-qvk-attff	0.7834	697.6907
exp-rte-adalora-qvk	0.7690	662.7145

name	acc_avg	train_runtime
exp-rte-adalora-qv-attff-intff	0.8123	775.0734
exp-rte-adalora-qv-intff-outff	0.8087	789.2883
exp-rte-adalora-qv-all-layers	0.8051	839.0118
exp-rte-adalora-qv-outff-attff	0.8051	750.2916
exp-rte-adalora-qv-intff	0.7978	739.0370
exp-rte-adalora-qv-outff	0.7870	703.3073
exp-rte-adalora-qv-attff	0.7726	655.3880
exp-rte-adalora-qv	0.7365	635.7862

name	acc_avg	train_runtime
exp-rte-adalora-vk-all-layers	0.8231	782.5745
exp-rte-adalora-vk-outff-attff	0.8195	697.7957
exp-rte-adalora-vk-attff-intff	0.8123	718.9266
exp-rte-adalora-vk-outff	0.8087	706.1469
exp-rte-adalora-vk-intff-outff	0.8051	795.1997
exp-rte-adalora-vk-intff	0.7906	730.3465
exp-rte-adalora-vk	0.7653	653.4077
exp-rte-adalora-vk-attff	0.7653	688.9600

name	acc_avg	train_runtime
exp-rte-adalora-v-all-layers	0.8195	746.3421
exp-rte-adalora-v-attff-intff	0.8123	685.0865
exp-rte-adalora-v-outff-attff	0.8087	662.9909
exp-rte-adalora-v-intff-outff	0.8087	759.4900
exp-rte-adalora-v-outff	0.8051	675.4465
exp-rte-adalora-v-intff	0.8014	647.7331
exp-rte-adalora-v	0.7690	567.2110
exp-rte-adalora-v-attff	0.7653	600.7299

ПРИЛОЖЕНИЕ 15

Сравнение методов

name	eval_f1	eval_acc	time	params	dataset
adalora-all-layers	0.987	0.982	197.213	2.03	cb
ia3	0.974	0.964	146.847	0.53	cb
lora-v-intff	0.966	0.982	148.307	1.29	cb
lora-all-layers	0.963	0.964	169.445	2.54	cb
adalora-v-intff	0.953	0.982	155.555	1.08	cb
fullft	0.918	0.934	234.158	100.00	cb

name	eval_acc	time	params	dataset
adalora-v-intff	0.801	675.901	1.08	rte
adalora-all-layers	0.798	839.888	2.03	rte
ia3	0.757	611.074	0.53	rte
lora-all-layers	0.728	735.463	2.54	rte
lora-v-intff	0.720	618.645	1.29	rte
fullft	0.658	834.507	100.00	rte

name	eval_acc	time	params	dataset
lora-v-intff	0.814	2598.102	1.29	boolq
adalora-all-layers	0.801	3240.465	2.03	boolq
adalora-v-intff	0.797	2706.861	1.08	boolq
lora-all-layers	0.797	2920.586	2.54	boolq
fullft	0.792	3231.104	100.00	boolq
ia3	0.782	2552.702	0.53	boolq

ПРИЛОЖЕНИЕ 16

Нагрузка на GPU

Тип	Параметры	GPU, MiB
FULLFT	batch_sz = 32	4800
FULLFT	batch_sz = 64	7680
FULLFT	batch_sz = 128	15000
LORA	batch_sz=32, r=16, tm:[q, v]	3000
LORA	batch_sz=64, r=16, tm:[q, v]	5060
LORA	batch_sz=64, r=4, tm:[q, v]	5000
LORA	batch_sz=64, r=1, tm:[q, v]	7490
LORA	batch_sz=64, r=16, tm:all-layers	7700
LORA	batch_sz=64, r=64, tm:all-layers	7900
LORA	batch_sz=128, r=16, tm:[q, v]	10300
IA3	batch_sz=64, tm: [v, k]	5000
IA3	batch_sz=64, tm: [v, k, attff, outff]	7440
IA3	batch_sz=128, tm: [v, k]	10310
ADALORA	batch_sz=64, init_r=12, tm:[q, v]	5080
ADALORA	batch_sz=64, tm:all-layers	7590
ADALORA	batch_sz=128, tm:[q, v]	10310